

Universo



Crear una aplicación web funcional en Angular que permita explorar, filtrar y gestionar una colección de videojuegos clásicos y modernos de la saga Mario y sus spin-offs, aplicando Standalone Components, Inyección de Dependencias con `inject()`, Signals, Señales Derivadas (`computed`), Efectos Secundarios (`effect`) y comunicación bidireccional mediante `input` y `output`.

Requisitos:

1. Estructura y Datos (JuegosService)

Crea un servicio global llamado `JuegosService` (con `providedIn: 'root'`) que actúe como el "cerebro" de la aplicación:

- Estado reactivo: Una signal privada que almacene un listado inicial de al menos 5 o 6 juegos reales (incluyendo título, plataforma, año de lanzamiento y si es juego principal o spin-off).
- Métodos públicos:
 - Un método para añadir un nuevo juego a la colección.
 - Un método para eliminar un juego de la lista por su identificador o índice.
- Reactividad Avanzada Obligatoria:
 - `computed()`: Crea una propiedad computada que calcule automáticamente el número total de juegos en el catálogo (o el conteo filtrado por categoría).
 - `effect()`: Utiliza un `effect()` para que, cada vez que la lista de juegos sufra modificaciones, se guarde automáticamente en el `localStorage` del navegador para que no se pierda al recargar la página.

2. Componente Hijo (JuegoItemComponent)

Crea un componente visual independiente para representar cada tarjeta de videojuego individual:

- Debe recibir los datos del juego desde el componente padre utilizando la función moderna `input.required()`.
- Debe mostrar de forma clara y atractiva el título, la plataforma y el tipo de juego (plataformas, kart, deportes, RPG...).
- Debe incluir un botón estético para eliminar el juego de la colección.

- Debe emitir un evento hacia arriba utilizando la función moderna `output()` cuando el usuario pulse el botón de eliminar.

3. Componente Padre (AppComponent)

El componente principal de la aplicación debe encargarse de:

- Inyectar el `JuegosService` utilizando la función moderna `inject()`.
- Mostrar en la cabecera el contador total de juegos en tiempo real gracias al `computed()` del servicio.
- Contener un formulario sencillo (inputs de texto para el título y la plataforma, y un botón) para permitir al usuario dar de alta nuevos títulos de forma interactiva.
- Renderizar el listado utilizando las estructuras de control nativas de Angular (`@for` y el bloque `@empty` para mostrar un mensaje si el catálogo se queda vacío).
- Integrar el componente hijo `<app-juego-item>` dentro del bucle, enlazando los datos con corchetes `[juego]` y escuchando la acción de borrado con paréntesis (`onEliminar`).

Será indispensable que se cumpla lo siguiente:

- La interfaz debe ser limpia, agradable y organizada (puedes añadir estilos personalizados en el CSS).
- La aplicación debe compilar y arrancar con `ng serve` sin arrojar ningún error en la consola.
- Comprueba que al añadir o borrar juegos, el contador se actualiza al instante y los cambios persisten en el almacenamiento local del navegador gracias al efecto reactivo.